

1 Introduction

What it makes clear: why this work matters and what the reader will get out of the document.

- High-level context (what process mining is, why it matters to organisations) — accessible, no deep technical detail yet.
- The motivation and a one-paragraph introduction of PM-MATE and the project's overall goal.
- Scope of the project and a short outline of the report (a reader's map of the chapters).
- *Red line:* foreshadow the goals that *Problem Conceptualization* sharpens and *Evaluation & Discussion* evaluates; the Introduction and Conclusion are bookends and should mirror each other.

Process mining has established itself as a discipline that turns the event data recorded by enterprise information systems into actionable insight about how processes actually run (W. van der Aalst, 2020). In recent years the field has undergone a paradigm shift: While early process mining was largely *isolated and backward-looking*, focussing on reconstructing past behaviour for retrospective analysis, the current generation of methods and tools strives to be *transparent, real-time, and forward-looking*, offering proactive and predictive support for ongoing operations (Grisold, van der Aa, et al., 2024; Reinkemeyer, 2020a). This evolution has raised both what organisations expect from process mining and what the surrounding research community seeks to deliver.

Around this shared subject, however, two communities have developed distinct priorities and toolsets. *Practitioners* are driven primarily by business value and return on investment. They require usable, scalable, well-integrated platforms that support sense-making and strategic alignment, and they tend to adopt commercial suites such as Celonis, SAP Signavio, and ARIS, whose strengths lie in usability and scalability (Grisold et al., 2020; W. van der Aalst, 2020). *Researchers*, by contrast, value transparency and extendibility: they develop new algorithms to improve precision measures, detect process dynamics, or address other complex technical challenges. And they need an environment in which their own modules can be implemented and tested, typically working with open, flexible toolkits such as ProM, PM4Py, bupaR, and RapidProM (Berti, Zelst, et al., 2019; Carmona & Sole, n.d.; van Dongen et al., 2005). The result is a divide between flexibility and research capability on one side and usability and scalability on the other.

This divide produces a practical gap. Advances that emerge in research, like context-aware sense-making, forward-looking support, the integration of natural language processing, or explainable AI, among others (Grisold, van der Aa, et al., 2024; Miksch et al., 2024; Thummarakoti, 2026), are often difficult to bring to bear on concrete business problems, because the tools in which they are prototyped are not the tools in which real processes are analysed. Commercial platforms are powerful but largely closed to external contributions, while research prototypes rarely reach the usability and scale that business adoption demands. Cutting-edge methods therefore reach industrial practice only slowly, if at all.

The project documented in this report, **PM-MATE**, responds directly to this gap. Its vision is to close the distance between scientific theory and business reality by providing a collaborative platform on which researchers can deploy their work as testable modules, enabling the immediate application of cutting-edge research to concrete challenges. In this way PM-MATE aims to combine the extendibility prized by researchers with the usability expected by practitioners, so that new process mining techniques can be trialled against real cases without leaving a single environment.

This report describes the motivation, conception, and realisation of PM-MATE, together with the research topics it is designed to host. The remainder is structured as follows. Chapter 2 introduces the concrete case that motivates the platform. Chapter 3 documents the organisation of the project. Chapter 4 establishes the theoretical background, looking into process mining, its platforms, and the specific topics the project engages, and Chapter 5 sets out the technical foundations and the system architecture. Chapter 6 conceptualises the problem into concrete objectives, Chapter 7 presents the implementation, and Chapter 8 evaluates and discusses the result. Chapter 9 concludes by revisiting the goals set out here.

2 The Case of PM-MATE

What it makes clear: the concrete, real situation that motivates the whole project.

- The setting/scenario PM-MATE is for, the practical problem or task, and who the stakeholders are.
- The expectations, requirements, and constraints that arise from the case.
- What "success" looks like from the case's perspective (sets up the criteria revisited in *Problem Conceptualization* and *Evaluation & Discussion*).
- The subsections (2.1 / 2.1.1 / 2.1.2 / 2.2) are placeholders — rename them after the natural parts of the case, e.g. *domain/background* → *the specific problem* → *stakeholders & requirements* → *scope*.
- *Red line*: this chapter sets the "demand." Everything later (which background you include, the problem definition, what you build, how you evaluate) should be traceable back to here.

2.1 First Subsection

2.1.1 First Subsection on Third Level

2.1.2 Second Subsection on Third Level

2.2 Second Subsection

3 Project Management

What it makes clear: that the project was run in a structured, professional way.

- Team roles and responsibilities; methodology (e.g. agile/Scrum, sprints, milestones).
- Timeline/schedule, task allocation, collaboration tools, decision-making, risk handling.
- Brief reflection on how the *process* went (lessons on working as a team).
- *Red line*: this is the "how we worked" track, deliberately separate from the technical narrative. Keep it self-contained — don't let it bleed into technical content.

The development of PM-MATE was organised around four areas of responsibility, coordinated under a common project-management and communication structure: *Development*, *UI/Design*, *Integration & Architecture*, and *Documentation*. Each area was assigned to one or two responsible team members who took ownership of its progress. At the same time, the team worked in a deliberately collaborative manner, members supporting one another across area boundaries wherever needed, so that being responsible for an area meant coordinating and driving it rather than working on it in isolation. This arrangement combined clear accountability with the flexibility to direct effort to wherever it was most needed at a given stage of the project.

Communication and collaboration were handled through two complementary channels. Microsoft Teams served as the central hub for structured collaboration. It was particularly used for sharing documents and other artefacts, and for staying in contact with the project supervisors. WhatsApp was used for quick, informal coordination within the team. Concentrating document exchange and supervisor communication in a single environment helped keep the current state of the work transparent and accessible to everyone involved.

The team maintained a regular meeting rhythm to synchronise progress and decisions. A weekly internal meeting was held once a week to review the state of each area, align on next steps, and resolve open questions. In addition, a weekly lunch offered a more formal occasion to stay connected as a whole team and keep the supervisors and requirements updated. Together, these recurring touchpoints kept the distributed responsibilities aligned and supported a steady flow of information throughout the project.

4 Theoretical Background

What it makes clear: the domain and scientific concepts a reader must know to follow the rest — establishes the shared vocabulary.

- Descriptive, literature-grounded; **no "we did..." and no problem-solving here** (that starts in *Problem Conceptualization*).
- **Golden rule:** include only background that is actually used later. If a concept never reappears, cut it.
- Distinguish from *Technical Background*: *Theoretical Background* = domain/scientific concepts; *Technical Background* = technical/engineering foundations.

4.1 Process Mining

defines the discipline: event logs; the discovery / conformance / enhancement triad; model-quality dimensions; object-centric process mining as a recent evolution. Makes clear what process mining is and its core techniques.

Process mining is a discipline situated between data science and process science that analyses *event data* to discover, monitor, and improve real processes (W. van der Aalst, 2016). Rather than relying on how a process is assumed to run, it derives knowledge from the digital traces that process executions leave behind in an organisation's information systems. Hence, process mining complements the predominantly model-driven perspective of classical business process management (Van Der Aalst et al., 2012).

The starting point for every process mining technique is the *event log*: a collection of cases (process instances), each a sequence of events that minimally record a case identifier, an activity label, and a timestamp, and are often enriched with further attributes such as the executing resource (Reinkemeyer, 2020b). From such a log, the literature distinguishes three principal types of analysis (Van Der Aalst et al., 2012) (W. van der Aalst, 2016). *Process discovery* derives a process model from an event log without any a priori model; *conformance checking* compares a model with a log to detect and quantify deviations between intended and observed behaviour; and *enhancement* repairs or extends an existing model using the log, for example by projecting performance information such as bottlenecks onto it. Because several models

can explain the same log, the quality of a discovered model is typically weighed along four competing dimensions: fitness, precision, generalisation, and simplicity (Van Der Aalst et al., 2012).

A more recent development, *object-centric process mining*, relaxes the traditional assumption that each event belongs to exactly one case, allowing processes in which several object types interact to be represented without distortion (W. M. P. van der Aalst, 2019). This and similar advances are increasingly absorbed into the integrated software environments (process mining platforms) examined in the following section.

4.2 Process Mining Platforms

what platforms are, what current platforms offer, what users see, and the vendor landscape (Gartner quadrant). Makes clear the practical software context PM-MATE sits in and the bar set by existing tools.

A *process mining platform* is the integrated software environment that makes the techniques of Section 4.1 usable in practice. It operationalises discovery, conformance checking, and enhancement as an end-to-end product, connecting to source systems such as ERP or CRM tools, extracting and maintaining event logs, and presenting the results for analysis (Thummarakoti, 2026). The central artefact a user works with is the discovered *process map*, complemented by a *variant explorer* that exposes how often each execution path occurs, dashboards of key performance indicators, conformance views that flag deviations from a reference model, and root-cause analyses that explain why inefficiencies arise (Zerbato et al., 2024).

Contemporary platforms have grown well beyond control-flow discovery into broad suites. Alongside the core analyses, they now commonly offer *task mining* (capturing desktop-level user interactions), process modelling and *simulation* or *digital-twin* capabilities, continuous monitoring and compliance checking, and *action* or automation engines that close the loop by triggering recommendations or downstream automation. Two newer capabilities are especially prominent: *object-centric process mining*, adopted to represent interacting object types (Celonis, for instance, builds its offering around an object-centric "intelligence graph"); and AI-assisted, conversational interfaces, increasingly framed as a way to supply the *operational context* that AI agents need in order to act on processes safely (2026 Gartner® Magic Quadrant™ for Process

Intelligence Platforms | Celonis, n.d.). This broadening is reflected in Gartner's renaming of its *Magic Quadrant* from *Process Mining* to *Process Intelligence Platforms*.



Figure 1 - Magic Quadrant for Process Intelligence Platforms 2026

That market is dominated by commercial vendors. In the April 2026 quadrant (Figure 1), the Leaders are Celonis, the most prominent pure-play vendor, together with SAP Signavio, ARIS, and Pegasystems, while many further vendors entered the field largely by acquiring specialist tools (for example UiPath, IBM, and Microsoft). Two caveats apply when reading the figure: it positions vendors by market standing rather than feature parity, and several capability areas, among them context knowledge; analysis recommendation; and temporal dynamics, remain active research directions, taken up in Section 4.3.

4.3 Topics in Process Mining

4.3.1 Context Knowledge

To be done: Freddi

4.3.2 Analysis Recommendation based on Process Logs

4.3.3 Temporal Dynamics

Business processes are commonly modeled as stable, analyzable systems that can be discovered and improved based on historical execution data. In practice, this assumption often does not hold, especially in the digital age. Processes change over time due to managerial interventions, regulatory shifts, evolving demand, or organizational learning (Grisold, Janiesch, et al., 2024). The changes can be triggered both by endogenous and exogenous events. This phenomenon, broadly known as dynamics, is studied through different lenses across research fields. In the context of process mining that our platform focuses on, it materializes as concept drift.

Bose et al., 2014) define concept drift as the situation in which a process changes while it is being analyzed. This undermines most classical process mining methods, which assume a stable underlying process: Two different iterations of a process which are analyzed as one through a single event log lead to misleading results. Drifts can be categorized and differentiated across different dimensions, such as duration, type, analysis and perspective. Therefore, methods that can recognize shifts in process behavior are required to ensure meaningful analysis and have been studied as the topic concept drift detection.

The literature commonly distinguishes four drift types: sudden (abrupt replacement of one process version by another), gradual (a transition period in which old and new versions coexist), incremental (step-by-step transformation through smaller changes), and recurring (reappearance of previously observed versions, for example due to seasonal patterns) Accordingly, Bose et al., 2014) identify three analytical tasks for concept drift detection. Change point detection (1) means detecting whether a process has changed and identifying when the change occurred. Change localization and characterization (2) refers to pinpointing the affected part of the process and describing the type of change, while change process discovery (3) aims at revealing the broader evolution of the process across multiple changes over time.

Regarding approaches to concept drift detection, (Sato et al., 2022) organize existing methods into drift detection from historical logs and online process mining for evolving

environments, noting that most approaches remain offline and control-flow-centric. (Elkhawaga et al., 2020) and (Adams et al., 2023) similarly show that no single method currently dominates across accuracy, latency, robustness, and scalability, meaning that method choice remains context-dependent. However, (Kraus & Van Der Aa, 2025) propose CV4CDD-4D as a machine learning-based approach for concept drift detection in business processes. The paper addresses limitations of existing techniques, especially their sensitivity to noise and lack of support for detection of more complex drift types. The central idea is to transform an event log into an image-based representation of the process sequence and then run a computer vision model to detect drift points and patterns automatically, allowing for the detection of all drift types and delivering state-of-the art performance.

For PM-MATE, this literature has two implications. First, process mining results cannot be assumed to reflect a single stable process, as undetected drift can distort models, metrics, and recommendations. Second, the literature suggests that process analysis in dynamic environments benefits from making process evolution over time visible and understandable. Temporal dynamics should thus be considered an important condition for producing meaningful and accurate insights from process mining techniques.

4.3.4 Process Complexity

Working on it (Nico)

Process complexity is a relevant concept in Business Process Management. The term describes the degree, to which a process is difficult to understand, analyze or explain (Cardoso, 2005). Business processes consist of a variety of activities, decision points, resources, data objects and more, that interact in various ways. The number of these elements, their interdependencies and their interactions are all influencing the complexity of a process (Cardoso et al., 2006).

The concept of process complexity is relevant because of its significant impact on organizational performance. High process complexity is associated with higher error rates, lower operational efficiency, reduced understandability and increased maintenance effort (Zhou et al., 2023). Therefore, organizations generally seek to design processes that are less complex, and consequently more efficient and manageable (...).

To this matter, measuring process complexity is an important prerequisite to enable

process analysis and streamlining initiatives. Complexity is influencing both quality and interpretability of discovered process models. Augusto et al. (2022) found that event log characteristics closely align with the quality of process models generated by process discovery algorithms. Similarly, increasingly complex process models become more difficult to understand for users (Mendling et al., 2007). Therefore, Complexity analysis can help to identify processes that may require simplification or redesign.

On top of model quality, process complexity can also be used to gain insight into process performance. Processes with high execution variation, frequent exceptions or a large number of execution paths are often times and indicator for inefficiencies or coordination challenges (Vidgof et al., 2023).

Traditional process complexity measures focus primarily on the structure of process models. However, while these metrics provide some useful insights, they are somewhat limited because they only describe the intended process design, rather than what the actual process execution looks like (Zhou et al., 2023). In practice, real process behaviour often times deviates from the modelled process, as it can be shown through process mining (...).

To address these limitations, research has recently increasingly focused on behavioural process complexity. This concept measures complexity based on event logs and observed process executions, rather than only the process models (Reviva & Aksu, 2023). It captures the actual behaviour of a process, which includes process variants, execution patterns and deviations that occur during execution (...).

This perspective is particularly relevant in Process Mining because event logs constitute the primary source of information used for process discovery and analysis (van der Aalst, 2016). Measuring complexity directly from event data therefore aligns naturally with the objectives of Process Mining. Compared to purely structural approaches, behavioral complexity provides a more realistic representation of how a process is executed in practice (Revina & Aksu, 2023; Vidgof & Mendling, 2023).

Measurements:

5 Technical Background

What it makes clear: the technical building blocks and the design of PM-MATE — the blueprint Implementation realises.

- The relevant technologies, frameworks, data formats/standards (e.g. event-log formats such as XES/OCEL), libraries, and tools — only those actually used.
- Technical concepts a reader needs to follow the Implementation.
- *Red line:* keep this at the level of *structure and design*; defer build details and how-it-was-coded to Implementation. Concepts (Ch *Theoretical Background*) $\neq$ technical foundations/design (Ch *Technical Background*) $\neq$ realisation (Ch *Implementation*) — keep the three boundaries clean to avoid duplication.

This chapter describes the technical foundations and design of PM-MATE, covering the technologies, frameworks, data standards, and architectural decisions that together define the blueprint realised in the Implementation chapter. This chapter is concerned with the engineering foundations on which PM-MATE is built. The discussion stays at the level of structure and design; the concrete realisation of each component is detailed in Chapter 7. We begin with the overall architecture and the authentication infrastructure it depends on, and then describe, in turn, the backend, the frontend, the data formats and storage, and the module system that makes the platform extensible.

5.1 Architecture

the high-level system architecture: components, how they interact, data flow (a diagram helps). Makes clear how PM-MATE is structured at a system level and the rationale behind the design.

PM-MATE is deployed as four cooperating containers managed with Docker Compose (Docker Inc., 2024). The two application services are the api service, which holds the process mining logic and the REST and WebSocket endpoints, and the web service, which renders the Next.js frontend in the browser. Alongside them, a Keycloak service provides OIDC-based identity management and is backed by a dedicated PostgreSQL database for persisting user accounts and realm configuration. The application services communicate over HTTP for ordinary requests and over WebSocket connections whenever the interface requires live updates, for instance while a long-running analysis is still in progress. All process mining data resides in a single bind-mounted directory owned by the api service, while Keycloak data is kept in its own Docker-managed volume.

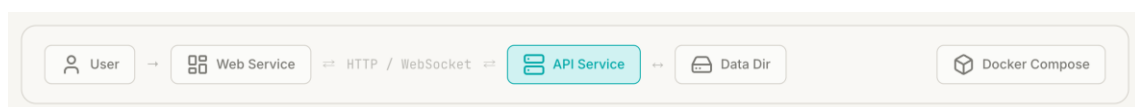


Figure 1: High-level architecture of PM-MATE

5.2 Authentication

User authentication in PM-MATE is handled through Keycloak, an open-source identity and access management service that runs as a dedicated container within the Docker Compose stack. Keycloak uses its own PostgreSQL database to persist user accounts, sessions, and realm configuration. The platform ships with a pre-configured realm that defines the application client, default user roles, and login flows. On production deployments, realm settings such as approved redirect URIs and the client secret are managed through the Keycloak administration console or via the bootstrap script provided with the repository.

The web application integrates Keycloak through Auth.js, which manages the OIDC redirect flow and issues sessions as signed JWTs stored in an encrypted cookie (Auth.js, 2024). On the api side, FastAPI validates incoming requests using PyJWT, fetching the realm's JWKS endpoint to verify token signatures without a database round-trip. Login is mandatory: both the frontend and the API enforce authentication on every route. User identities are keyed by the Keycloak subject claim, which isolates each user's event logs, jobs, AI configuration, and module settings from those of all other users.

5.3 Backend

The backend is written in Python, the language the team knows best and the de facto language of the data-science and process-mining ecosystems, which made the necessary libraries readily available (Python Software Foundation, 2025). Dependencies and virtual environments are managed with uv, a fast package and project manager written in Rust that replaces tools such as pip, poetry, and pyenv and lets every developer reproduce the same environment from a single specification (Astral, 2025). Code quality is maintained with Ruff for linting and formatting and Pyright for static type checking (Astral, 2024; Microsoft, 2024).

The API itself is built with FastAPI, a Python web framework that derives its interface from standard type annotations and produces an OpenAPI description of the entire API as a by-product (Ramirez, 2024). FastAPI is asynchronous by design, which shapes how the backend handles process mining work. Because such computations can be heavy and long running, the design separates request handling from computation: a request places work on an asynchronous job queue, returns immediately, and the queued work is processed in the background. Progress is reported to the browser over WebSocket connections, a full-duplex channel that allows the server to push updates without being

polled (Fette & Melnikov, 2011). Incoming and outgoing data are validated with Pydantic, whose definitions feed directly into the OpenAPI description mentioned above (Pydantic, 2024).

Process mining capabilities are provided by pm4py, an open-source library that exposes a broad collection of process mining algorithms in Python and reads the standard event-log formats described in the data section (Berti, van Zelst, et al., 2019). Building on pm4py means the platform can rely on tested implementations of the discovery and conformance techniques introduced in Chapter 4, rather than reimplementing them from scratch. Schema changes to the SQLite metadata database are managed with Alembic, which tracks and applies incremental migrations and keeps the schema versioned alongside the source code.

5.4 Frontend

The frontend is a Next.js application built with the App Router and React Server Components (Meta Open Source, 2024; Vercel, 2024). Next.js provides routing, server-side rendering, and a clear project structure on top of React, while React Server Components allow parts of the interface to render on the server and stream to the client, keeping the JavaScript bundle small. The application is written in TypeScript (Bierman et al., 2014; Microsoft, 2024), reflecting the same preference for compile-time error detection applied on the backend.

The visual layer combines Tailwind CSS, a utility-first styling framework, with shadcn/ui, a set of accessible React components that are copied into the project and can therefore be freely adapted (shadcn, 2024; Tailwind Labs, 2024). Application state is divided into two kinds: data fetched from the backend is managed by TanStack Query, which handles fetching, caching, and freshness, while purely local interface state is kept in Zustand (Poimandres, 2024; TanStack, 2024). TypeScript types are generated from the backend’s OpenAPI description and validated at runtime with Zod (McDonald, 2024; OpenAPI Initiative, 2021), ensuring that the frontend is compiled against the same contract the backend enforces.

Visualisation is handled by three libraries. Recharts and ECharts render the statistical charts, with ECharts reserved for the more complex interactive ones (Li et al., 2018; Recharts Group, 2024). Discovered process models, including directly-follows graphs, Petri nets, and process trees, are drawn with React Flow, a library designed for node-and-edge diagrams (xyflow, 2024).

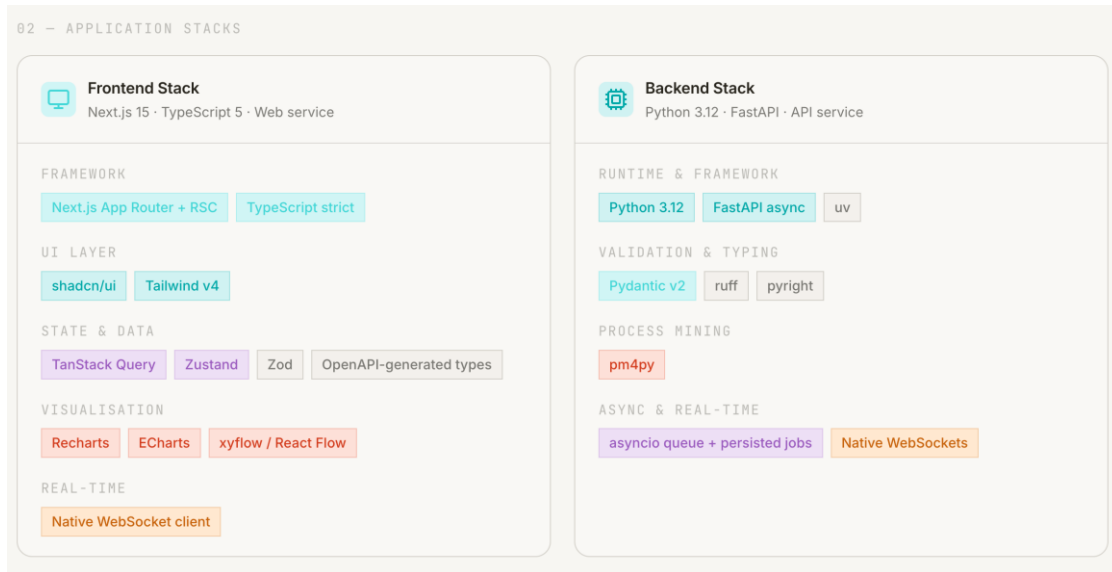


Figure 3: The application stacks of the two services: the frontend (left) and the backend (right).

5.5 Data Formats and storage

PM-MATE accepts event logs in XES format, including its compressed variant (.xes.gz), and CSV. XES is the IEEE 1849 standard for process mining event logs and serves as the primary input format; CSV is accepted for simpler tabular data, with a column-mapping step at import. A secondary XML import path is also available for advanced users. All formats are processed through pm4py. The platform is architected to be forward-compatible with OCEL 2.0, the object-centric event-log format introduced in Section 4.1: the internal storage layout reserves the necessary structures on disk, so extending ingestion to OCEL files requires no change to the core storage layer (Ghahfarokhi et al., 2021).

Internally, PM-MATE assigns each kind of data to the storage technology that suits it best, rather than forcing everything into a single database (Figure 4). Event logs and cached query results are held as Apache Parquet, a columnar file format well suited to data that is written once and then read column by column (Apache Software Foundation, 2024). DuckDB, an in-process analytical database, queries these Parquet files directly and provides modules with a full SQL interface over the event logs without loading them into memory first (Raasveldt & Mühleisen, 2019). SQLite handles the platform's own transactional state, namely application settings, job-queue status, and user metadata, with full ACID guarantees and zero configuration (SQLite, 2024). Keycloak maintains a separate PostgreSQL database for user accounts and realm data; this is intentionally kept apart from the application data stores. Finally, uploaded event-log files are kept

untouched in a dedicated directory, ensuring that the original source data is always traceable.



Figure 4: The persistence and analytics layer

5.6 The module system

The feature that most distinguishes PM-MATE is that it is designed to be extended by researchers and developers without requiring changes to the platform core. This is achieved through a filesystem-discovered module system, illustrated in Figure 5: each analysis capability lives in its own self-contained directory, which the platform discovers automatically at startup. A module declares itself in a manifest file (`manifest.yaml`) stating its name, version, category, capabilities, and declared dependencies, and provides its logic in a Python entry point that subclasses the `Module` base class from the platform's `module-sdk-py` package. Each module carries its own virtual environment, allowing it to depend on the exact library versions it needs without conflicting with the platform or with other modules. A companion TypeScript package, `module-sdk-ts`, is available for modules that contribute a frontend panel, giving frontend authors the same structured contract as Python authors on the backend.

Modules do not interact with one another directly. On the backend, they communicate through an event bus, which is a publish/subscribe channel with a capability registry managed by the `api` service. On the frontend, a module can contribute a panel that the web service renders. The interface between a module and the platform is a wrapper that supplies each module with a context object, through which the module receives the event log in whichever representation suits it: either a data frame, a `pm4py` event log, or a SQL-queryable view, while the platform retains full control over how data is stored and retrieved. In addition to the modules shipped with the platform, users can install additional modules by uploading a zip archive or supplying a git URL through the application settings. Module installations are tracked per user, so each user manages their own active set independently; a `Restore defaults` action re-adds any bundled modules a user has removed.

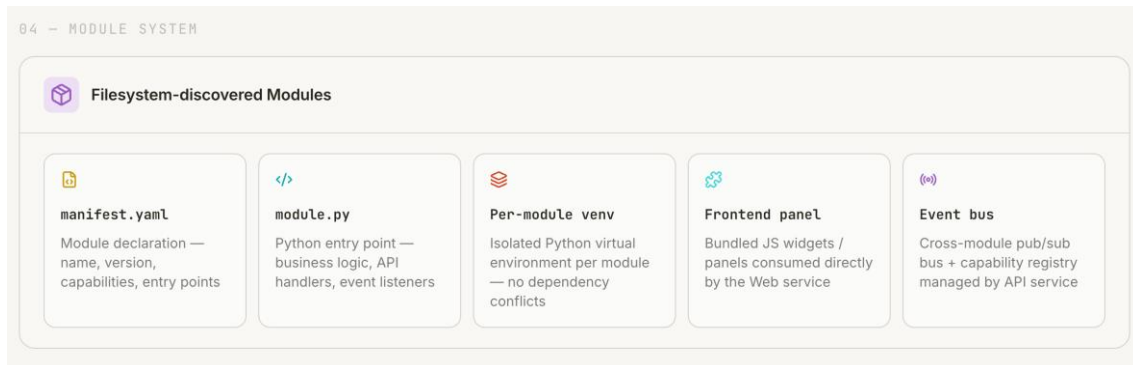


Figure 5: The building blocks of a module

5.7 Mate AI

PM-MATE includes a built-in AI assistant, Mate AI, accessible as a persistent chat panel within the interface. Rather than coupling the platform to a specific LLM provider, the assistant is designed to work with any OpenAI-compatible endpoint: users supply their own API key, select the model, and optionally customise the system prompt through the application settings. Keys and configuration are stored in the local SQLite metadata database and never transmitted outside the host. The assistant is context-aware: it can reference the currently active event log, the results of recent analysis jobs, and the set of enabled modules, which allows it to provide targeted guidance throughout a process analysis session.

References

- 2026 Gartner® Magic Quadrant™ for Process Intelligence Platforms | Celonis. (n.d.). Retrieved 18 June 2026, from <https://www.celonis.com/insights/reports/gartner-magic-quadrant>
- Adams, J. N., Pitsch, C., Brockhoff, T., & Van Der Aalst, W. M. P. (2023). An Experimental Evaluation of Process Concept Drift Detection. *Proceedings of the VLDB Endowment*, 16(8), 1856–1869. <https://doi.org/10.14778/3594512.3594517>
- Apache Software Foundation. (2024). *Apache Parquet documentation*. <https://parquet.apache.org/docs/>
- Astral. (2024). *Ruff: An extremely fast Python linter and code formatter*. <https://docs.astral.sh/ruff/>
- Astral. (2025). *uv: An extremely fast Python package and project manager*. <https://docs.astral.sh/uv/>
- Auth.js. (2024). *Auth.js (NextAuth.js v5)*. <https://authjs.dev/>
- Berti, A., van Zelst, S. J., & van der Aalst, W. M. P. (2019). Process Mining for Python (PM4Py): Bridging the Gap Between Process- and Data Science. *arXiv Preprint arXiv:1905.06169*. <https://arxiv.org/abs/1905.06169>
- Berti, A., Zelst, S. J. van, & Aalst, W. van der. (2019). *Process Mining for Python (PM4Py): Bridging the Gap Between Process- and Data Science* (arXiv:1905.06169). arXiv. <https://doi.org/10.48550/arXiv.1905.06169>
- Bierman, G., Abadi, M., & Torgersen, M. (2014). *Understanding TypeScript. Lecture Notes in Computer Science, Vol. 8586*, 257–281. https://doi.org/10.1007/978-3-662-44202-9_11
- Bose, R. P. J. C., Van Der Aalst, W. M. P., Zliobaite, I., & Pechenizkiy, M. (2014). Dealing With Concept Drifts in Process Mining. *IEEE Transactions on Neural Networks and Learning Systems*, 25(1), 154–171. <https://doi.org/10.1109/TNNLS.2013.2278313>
- Carmona, J., & Sole, M. (n.d.). *PMLAB: An Scripting Environment for Process Mining*.
- Docker Inc. (2024). *Overview of Docker Compose*. Docker Documentation. <https://docs.docker.com/compose/>
- Elkhawaga, G., Abuelkheir, M., Barakat, S. I., Riad, A. M., & Reichert, M. (2020). CONDA-PM—A Systematic Review and Framework for Concept Drift Analysis in Process Mining. *Algorithms*, 13(7), 161. <https://doi.org/10.3390/a13070161>
- Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol* (RFC 6455). Internet Engineering Task Force (IETF). <https://www.rfc-editor.org/rfc/rfc6455>

- Ghahfarokhi, A. F., Park, G., Berti, A., & van der Aalst, W. M. P. (2021). OCEL: A Standard for Object-Centric Event Logs. In L. Bellatreche & others (Eds), *New Trends in Database and Information Systems* (Vol. 1450, pp. 169–175). Springer. https://doi.org/10.1007/978-3-030-85082-1_16
- Grisold, T., Janiesch, C., Röglinger, M., & Wynn, M. T. (2024). Managing Dynamics in and Around Business Processes. *Business & Information Systems Engineering*, 66(5), 533–540. <https://doi.org/10.1007/s12599-024-00895-2>
- Grisold, T., Mendling, J., Otto, M., & Vom Brocke, J. (2020). Adoption, use and management of process mining in practice. *Business Process Management Journal*, 27(2), 369–387. <https://doi.org/10.1108/BPMJ-03-2020-0112>
- Grisold, T., van der Aa, H., Franzoi, S., Hartl, S., Mendling, J., & Brocke, J. V. (2024). A Context Framework for Sense-making of Process Mining Results. *2024 6th International Conference on Process Mining (ICPM)*, 57–64. <https://doi.org/10.1109/ICPM63005.2024.10680665>
- Kraus, A., & Van Der Aa, H. (2025). Machine learning-based detection of concept drift in business processes. *Process Science*, 2(1), 5. <https://doi.org/10.1007/s44311-025-00012-w>
- Li, D., Mei, H., Shen, Y., Su, S., Zhang, W., Wang, J., Zu, M., & Chen, W. (2018). ECharts: A declarative framework for rapid construction of web-based visualization. *Visual Informatics*, 2(2), 136–146. <https://doi.org/10.1016/j.visinf.2018.04.011>
- McDonald, C. (2024). *Zod documentation*. <https://zod.dev/>
- Meta Open Source. (2024). *Server Components*. React Documentation. <https://react.dev/reference/rsc/server-components>
- Microsoft. (2024). *The TypeScript handbook*. <https://www.typescriptlang.org/docs/handbook/intro.html>
- Miksch, S., Di Ciccio, C., Soffer, P., & Weber, B. (2024). Visual Analytics Meets Process Mining: Challenges and Opportunities. *IEEE Computer Graphics and Applications*, 44(6), 132–141. <https://doi.org/10.1109/MCG.2024.3456916>
- OpenAPI Initiative. (2021). *OpenAPI Specification (Version 3.1.0)*. <https://spec.openapis.org/oas/latest.html>
- Poimandres. (2024). *Zustand documentation*. <https://zustand.docs.pmnd.rs/>
- Pydantic. (2024). *Pydantic (Version 2) documentation*. <https://docs.pydantic.dev/>
- Python Software Foundation. (2025). *The Python language reference*. <https://docs.python.org/3/>

- Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An Embeddable Analytical Database. *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data*, 1981–1984.
<https://doi.org/10.1145/3299869.3320212>
- Ramirez, S. (2024). *FastAPI documentation*. <https://fastapi.tiangolo.com/>
- Recharts Group. (2024). *Recharts documentation*. <https://recharts.org/>
- Reinkemeyer, L. (2020a). Business View: Towards a Digital Enabled Organization. In L. Reinkemeyer (Ed.), *Process Mining in Action: Principles, Use Cases and Outlook* (pp. 197–206). Springer International Publishing. https://doi.org/10.1007/978-3-030-40172-6_22
- Reinkemeyer, L. (2020b). Process Mining in a Nutshell. In L. Reinkemeyer (Ed.), *Process Mining in Action: Principles, Use Cases and Outlook* (pp. 3–10). Springer International Publishing.
https://doi.org/10.1007/978-3-030-40172-6_1
- Sato, D. M. V., De Freitas, S. C., Barddal, J. P., & Scalabrin, E. E. (2022). A Survey on Concept Drift in Process Mining. *ACM Computing Surveys*, 54(9), 1–38. <https://doi.org/10.1145/3472752>
- shadcn. (2024). *Shadcn/ui documentation*. <https://ui.shadcn.com/docs>
- SQLite. (2024). *SQLite: Features*. <https://sqlite.org/features.html>
- Tailwind Labs. (2024). *Tailwind CSS documentation*. <https://tailwindcss.com/docs>
- TanStack. (2024). *TanStack Query documentation*. <https://tanstack.com/query/latest>
- Thummarakoti, S. (2026). Transforming Business Processes with Process Mining and Cloud Integration: Applications, Challenges, and Innovations. In J. C. Bansal, S. Borah, S. Hussain, & S. Salhi (Eds), *Computing and Machine Learning* (pp. 477–488). Springer Nature.
https://doi.org/10.1007/978-981-95-2872-1_34
- van der Aalst, W. (2016). Data Science in Action. In W. van der Aalst (Ed.), *Process Mining: Data Science in Action* (pp. 3–23). Springer. https://doi.org/10.1007/978-3-662-49851-4_1
- van der Aalst, W. (2020). Academic View: Development of the Process Mining Discipline. In L. Reinkemeyer (Ed.), *Process Mining in Action: Principles, Use Cases and Outlook* (pp. 181–196). Springer International Publishing. https://doi.org/10.1007/978-3-030-40172-6_21
- Van Der Aalst, W., Adriansyah, A., De Medeiros, A. K. A., Arcieri, F., Baier, T., Blickle, T., Bose, J. C., Van Den Brand, P., Brandtjen, R., Buijs, J., Burattin, A., Carmona, J., Castellanos, M., Claes, J., Cook, J., Costantini, N., Curbera, F., Damiani, E., De Leoni, M., ... Wynn, M. (2012). Process Mining Manifesto. In F. Daniel, K. Barkaoui, & S. Dustdar (Eds), *Business Process*

Management Workshops (Vol. 99, pp. 169–194). Springer Berlin Heidelberg.

https://doi.org/10.1007/978-3-642-28108-2_19

- van der Aalst, W. M. P. (2019). Object-Centric Process Mining: Dealing with Divergence and Convergence in Event Data. In P. C. Ölveczky & G. Salaün (Eds), *Software Engineering and Formal Methods* (pp. 3–25). Springer International Publishing. https://doi.org/10.1007/978-3-030-30446-1_1
- van Dongen, B. F., de Medeiros, A. K. A., Verbeek, H. M. W., Weijters, A. J. M. M., & van der Aalst, W. M. P. (2005). The ProM Framework: A New Era in Process Mining Tool Support. In G. Ciardo & P. Darondeau (Eds), *Applications and Theory of Petri Nets 2005* (pp. 444–454). Springer. https://doi.org/10.1007/11494744_25
- Vercel. (2024). *Next.js documentation*. <https://nextjs.org/docs>
- xyflow. (2024). *React Flow documentation*. <https://reactflow.dev/>
- Zerbato, F., Zimmermann, L., Vrotsou, K., & Weber, B. (2024). *From Analysis to Findings: How Do Process Mining Analysts Discover Results?* SSRN. <https://doi.org/10.2139/ssrn.4975616>